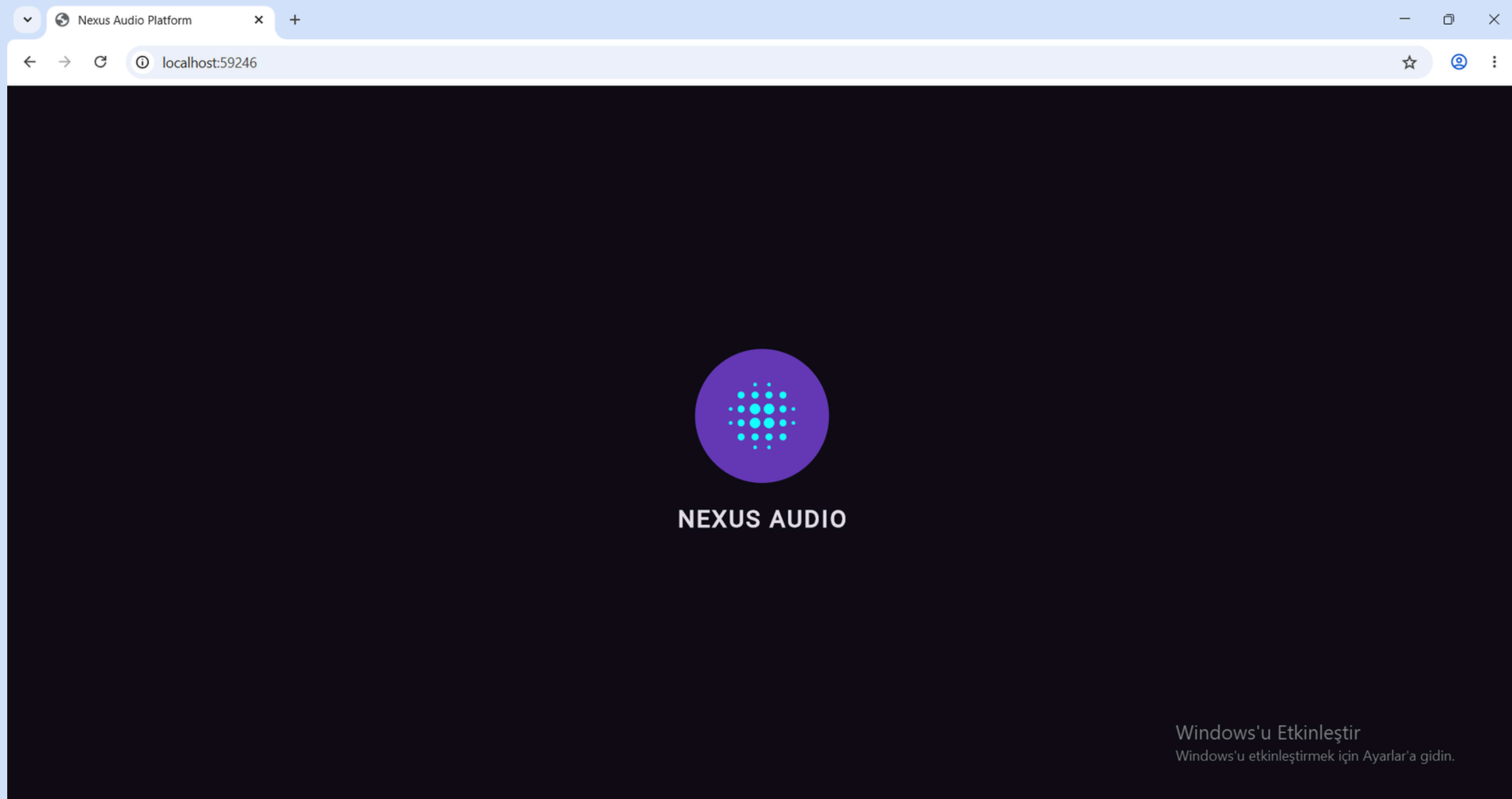
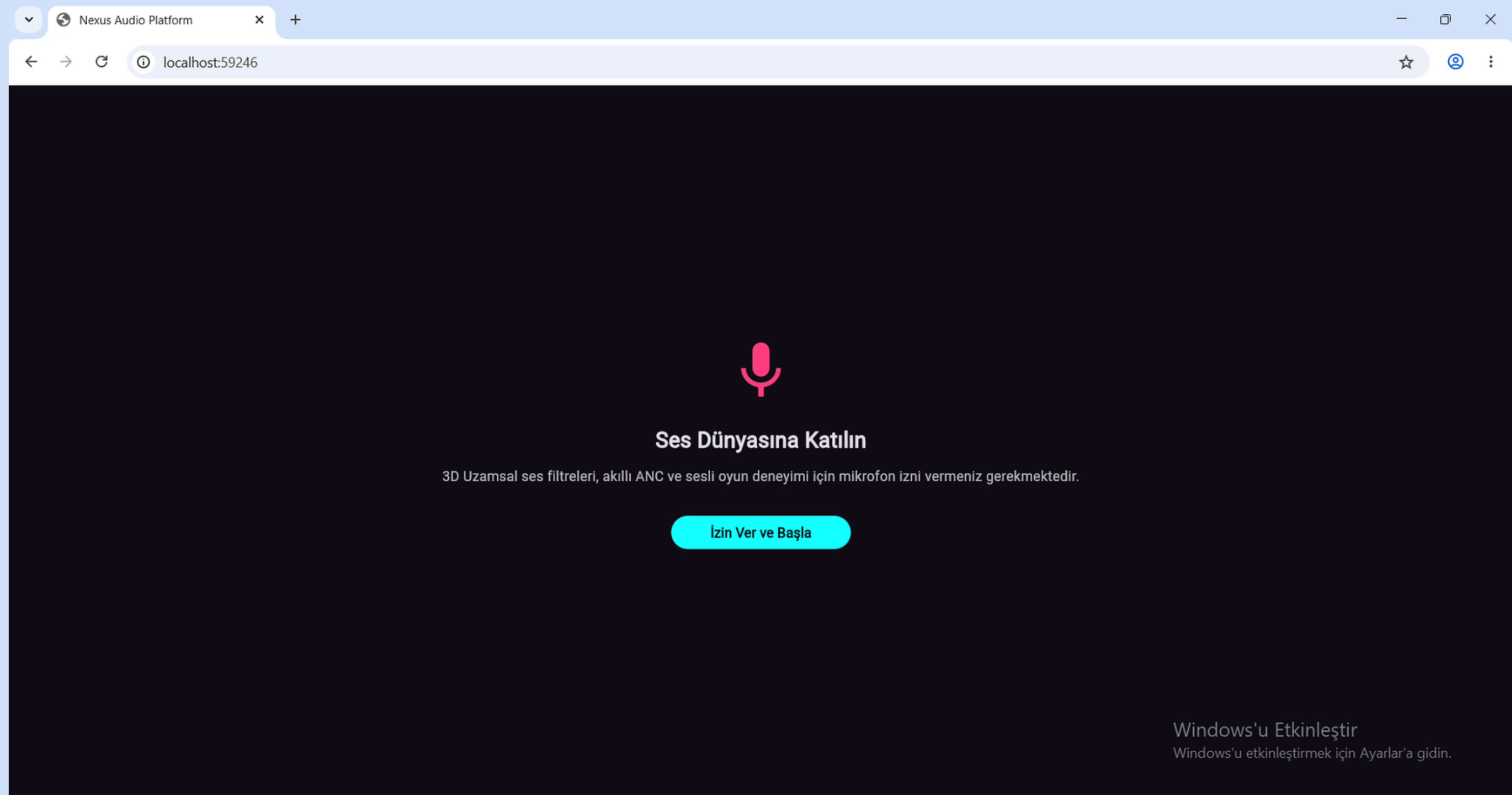
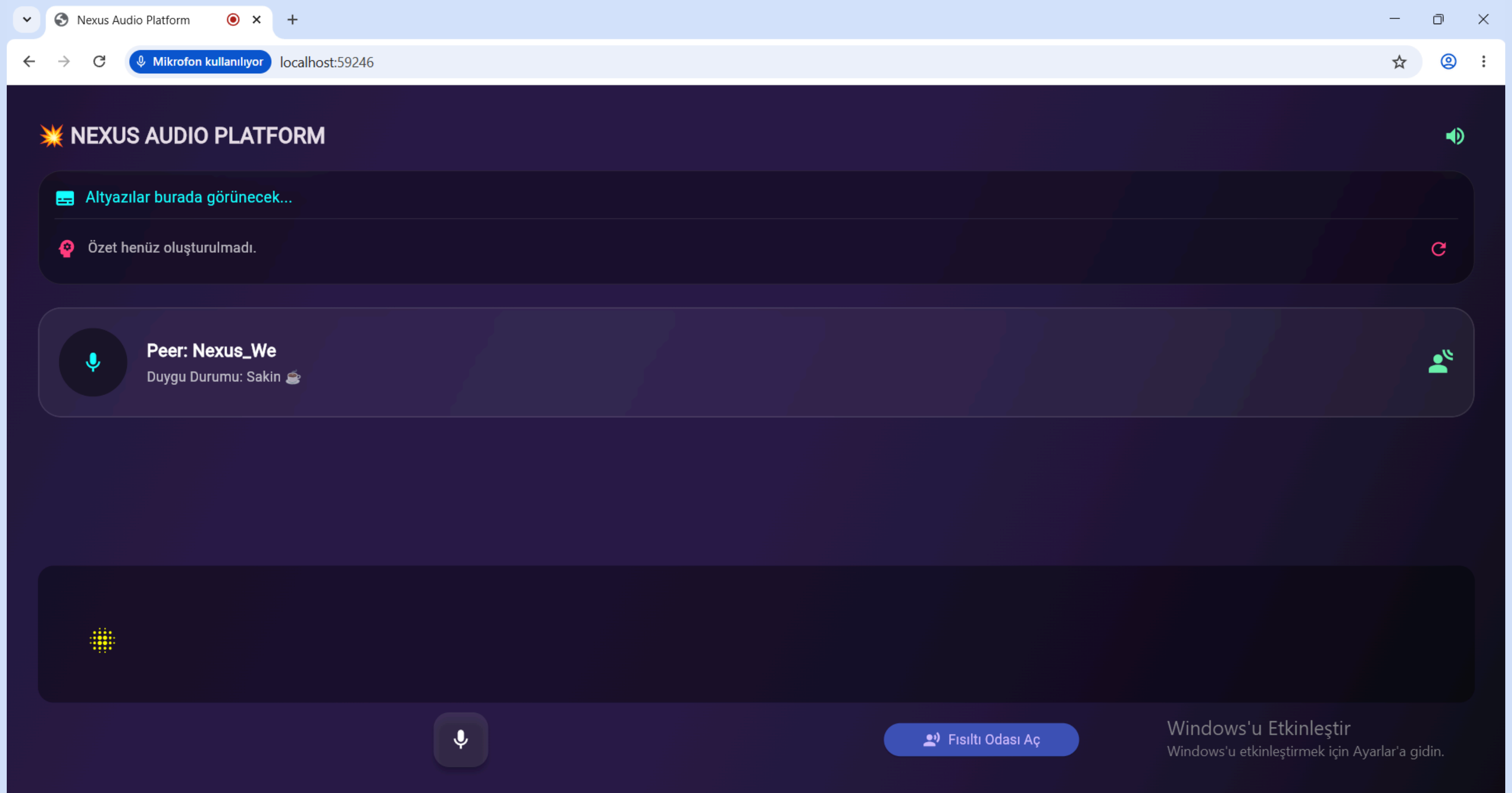




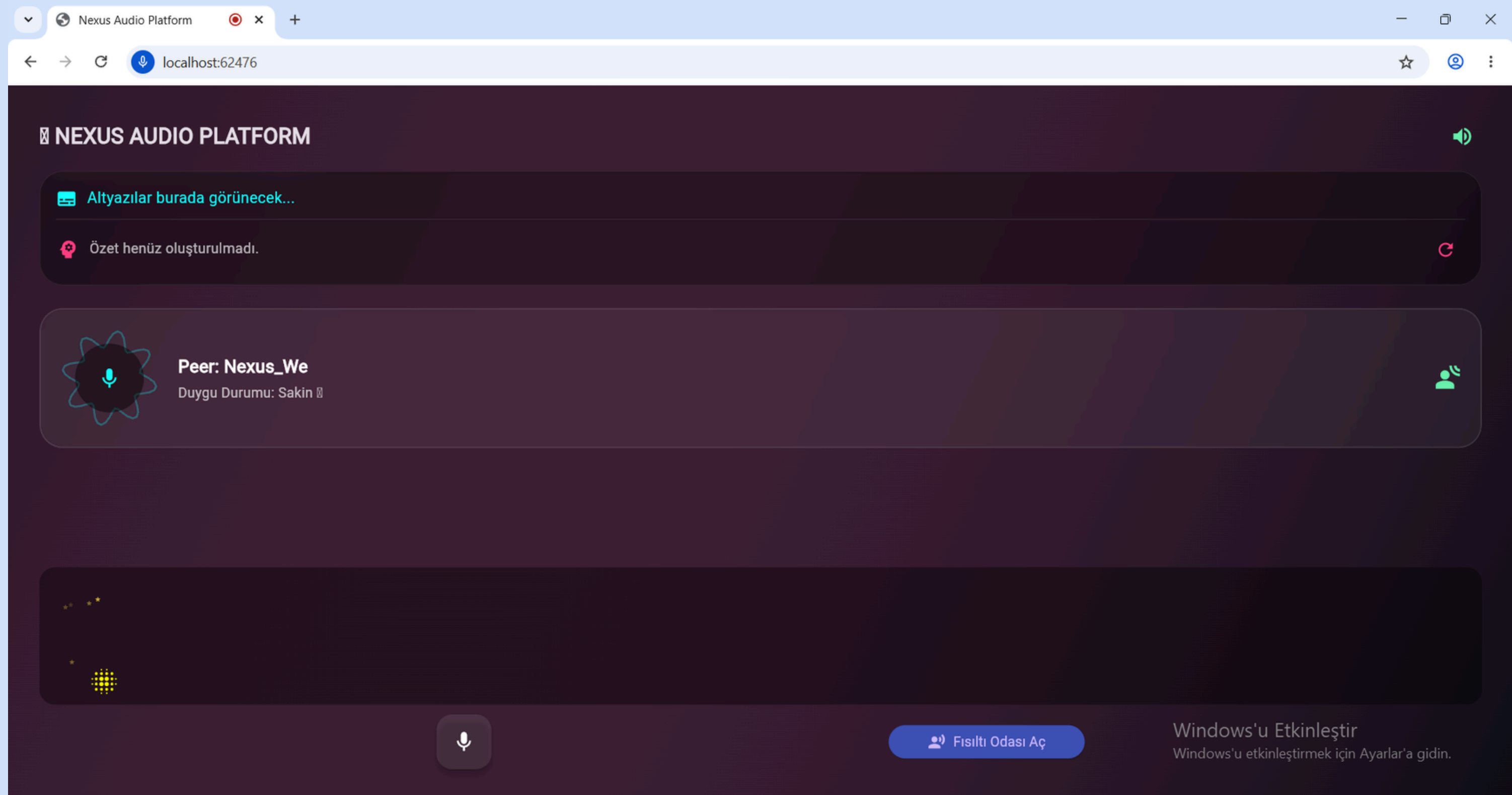
Uygulamanın asenkron başlatma süreçlerini (örneğin bellek optimizasyonları ve soket hazırlıkları) yöneten bir ön katman tasarlandı. Kullanıcı deneyimini artırmak amacıyla, donanım kaynaklarını tüketmeyen hafif bir SingleTickerProviderStateMixin mimarisi kuruldu. Bu mimari üzerinden, ses dalgalarını simüle eden 2 saniyelik dinamik bir nabız (pulse) animasyonu kodlandı.



Uygulamanın çalışması için kritik öneme sahip olan mikrofon erişimi, `permission_handler` kütüphanesiyle asenkron bir boru hattına (`_checkInitialPermissions`) bağlandı. Kullanıcıyı doğrudan sistem diyaloguyla boğmamak adına, işletim sistemi düzeyindeki izin durumunu arka planda sorgulayan kontrol mekanizması tamamlandı. İzin verilmediği senaryolarda devreye giren ve kullanıcıya uzamsal ses filtreleri ile akıllı gürültü engelleme (ANC) süreçlerini açıklayan bilgilendirici bir onay katmanı (`AnimatedSwitcher`) arayüze entegre edildi.

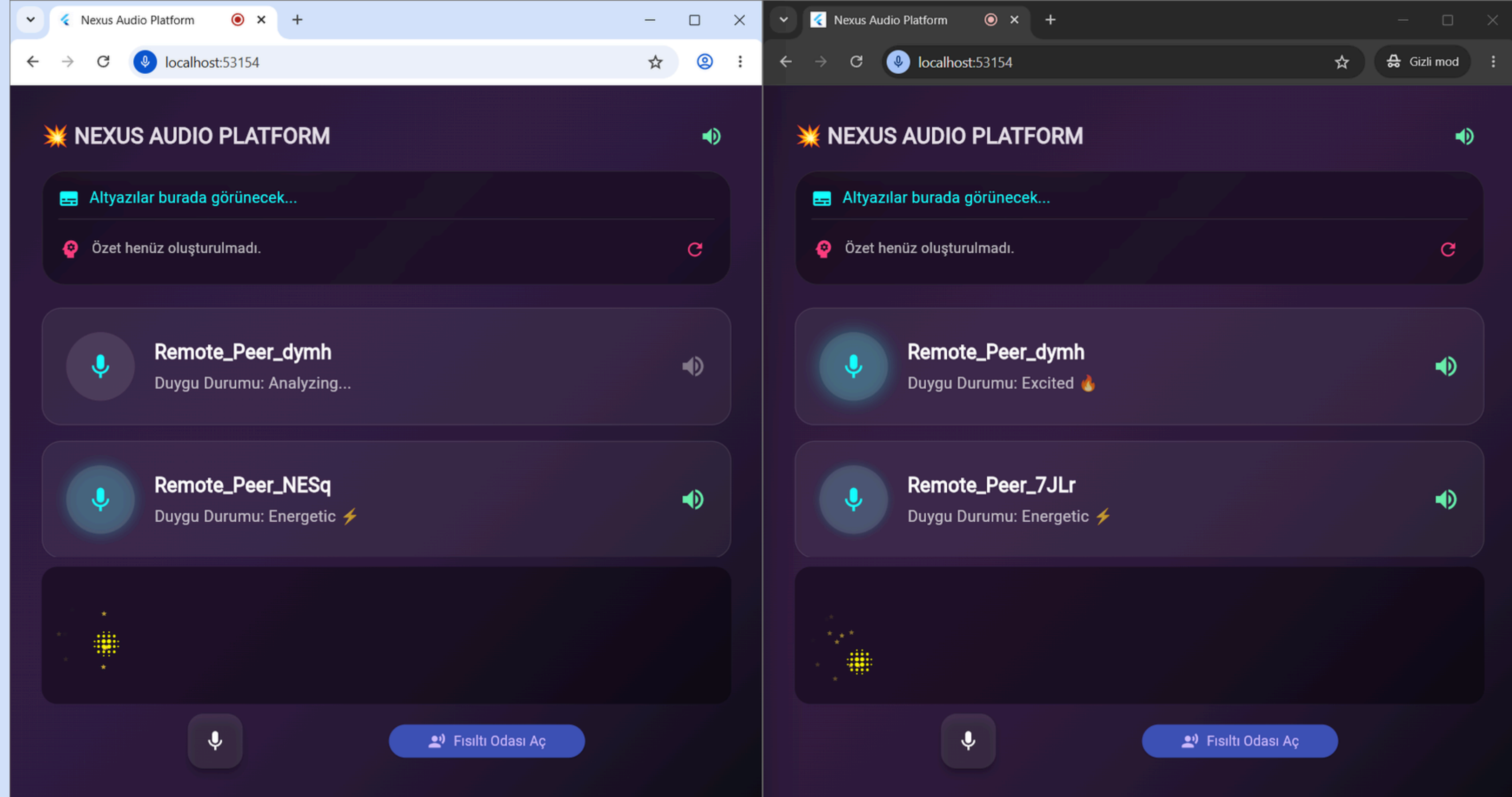


SocketService üzerinden merkezî sinyalleşme sunucusuna güvenli bir şekilde join-room isteği gönderilerek canlı oda bağlantısı sağlandı. Kullanıcının lokal mikrofonundan alınan ses akışı, gerçek zamanlı desibel seviyesi analizine bağlandı. Bu ses yoğunluğuna (voiceLevel) bağlı olarak dinamik parçacıklar üreten bir sistem simüle edildi. Konuşma esnasında ses eşiği yükseldikçe karakterin dikey ekseninde hareket etmesi sağlanarak, sesin sisteme başarıyla ulaştığına dair kullanıcıya oyunlaştırılmış bir görsel geri bildirim mekanizması sunuldu. En altta yer alan butonlar ile WebRTC medya akışındaki ses hatlarının asenkron olarak kapatılıp açılabilmesi (\_toggleMute) ve fısıltı odası geçişleri kontrol altına alındı.

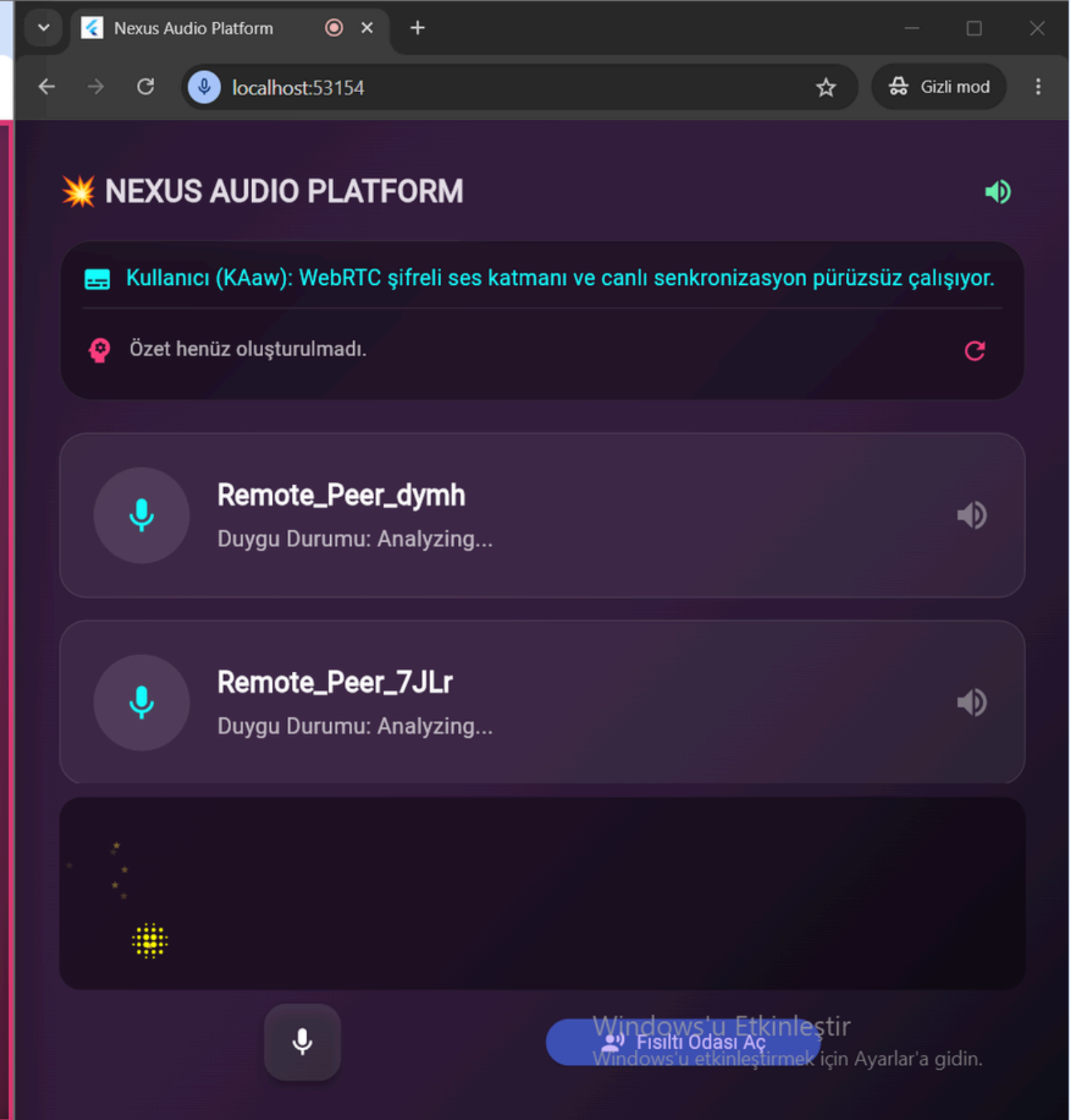
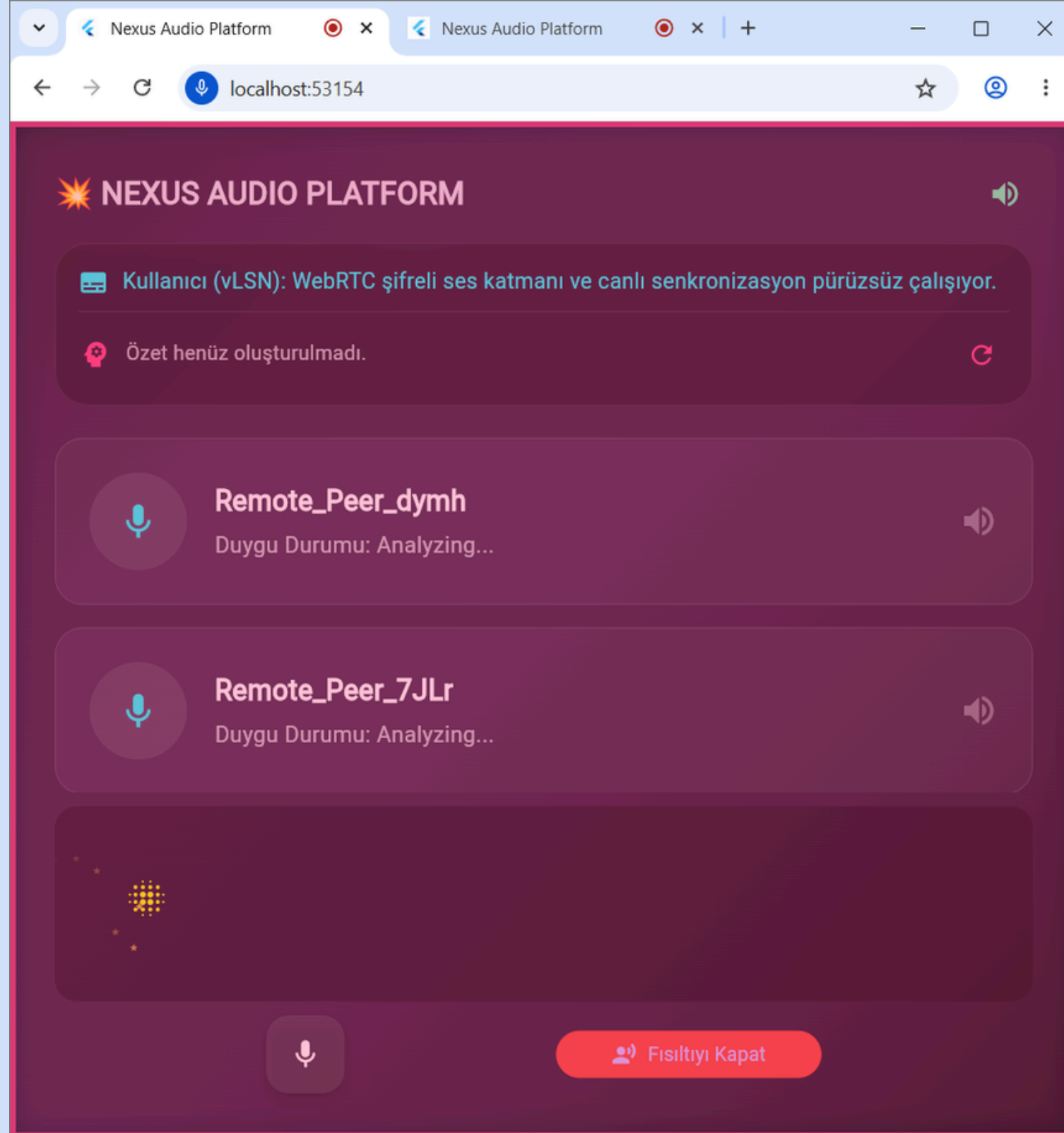


İstemci tarafında Timer.periodic ile her 150 milisaniyede bir mikrofondan örnekleme alan bir lokal motor simülasyonu kuruldu. Bu motor, kullanıcının ses dalgalarını analiz ederek duygu durumunu belirlemektedir. Yakalanan veriler voice-analytics-update soket olayı üzerinden sunucuya iletilmekte, sunucu ise odadaki diğer eşleri anlık olarak besleyerek herkesin duygu durumunun (Örn: Enerjik ⚡, Gergin/Yüksek 🔥 veya Analiz Ediliyor...) gecikmesiz olarak listelenmesini (Peer Analytics Management) sağlamaktadır.





Odadaki genel ses akışından tamamen bağımsız, seçilen hedef partnerlerle birebir şifreli görüşme yapılabilmesini sağlayan "Secure Whisper" mimarisi geliştirildi. Kullanıcı "Güvenli Fısıltı Aç" butonuna bastığı an, sunucu tarafında start-whisper alt grubu oluşturulmaktadır. Genel WebRTC havuzundan izole edilen ve sadece bu fısıltı kanalına ait olan kriptografik \_whisperConnections ses köprüsü kurulmaktadır. Fısıltı modu aktif olduğunda, kullanıcının gizli siber kanalda olduğunu hatırlatmak amacıyla ekran sınırlarına parıltılı pembe renkli görsel sınırlar eklenmiştir.



Web ve Chrome tarayıcılarında ses motorunun donanımı boşa çıkaramaması veya döngüsel çakışmalara girmesini engellemek amacıyla \_sttLock adında özel bir boolean emniyet kilidi algoritması geliştirildi. speech\_to\_text kütüphanesiyle entegre çalışan bu kilit sayesinde, ses yakalama işlemi tamamlandığında donanım güvenli bir şekilde serbest bırakılmakta ve konuşmalar anlık olarak metne (audio-transcription) dönüştürülmektedir. Sunucu katmanına iletilen bu transkripsiyonlar, seçilen hedef dile göre (TR, EN, DE, RU) işlenerek odadaki tüm istemcilere canlı alt yazı (live-subtitle) olarak basılmaktadır.